94.3
Pass

Tanmay

PDF generated at: 17 Feb 2026 03:40:43 UTC

View this report on HackerRank

Score

94.3% • 165.0 / 175
scored in eClerx - Java Coding Test - 3+ Years New in 83 min 4 sec on 16 Feb 2026 20:58:32 IST

Candidate Information

Email	tanmaymhatre40@gmail.com
Test	eClerx - Java Coding Test - 3+ Years New
Candidate Packet	View
Taken on	16 Feb 2026 20:58:32 IST
Time taken	83 min 4 sec/ 90 min
Work Experience	5 years
Invited by	Vipul
Invited on	13 Feb 2026 18:58:57 IST

Skill Distribution

No.	Skill	Score
1	Spring Boot Advanced	50%

2	Spring Boot Intermediate	100%
3	Problem Solving Intermediate	100%
4	Java Intermediate	100%

Tags Distribution

Spring Boot	50%	Hard	50%
TDD	50%	Medium	100%
Spring	100%	Graphs	100%
Java	100%	Exceptions	100%
OOP	100%		

Questions

Spring Boot (Advanced) • 15 / 25

No.	Question	Skill	Score	Code Quality	Time Taken	Status
1	Specification Multiple Choice	Spring Boot (Advanced)	0/5	-	1 min	⊗

2	Cracker Multiple Choice	Spring Boot (Advanced)	5/5	-	4 min 35 sec	✓
3	Output Multiple Choice	Spring Boot (Advanced)	0/5	-	3 min 29 sec	✗
4	Response Multiple Choice	Spring Boot (Advanced)	5/5	-	2 min 43 sec	✓
5	Spring Scopes Multiple Choice	Spring Boot (Intermediate)	5/5	-	2 min 14 sec	✓

Problem Solving (Intermediate) • 75 / 75

No.	Question	Skill	Score	Code Quality	Time Taken	Status
6	Minimize Path Value Coding	Problem Solving (Intermediate)	75/75	-	33 min 38 sec	✓

Java (Intermediate) • 75 / 75

No.	Question	Skill	Score	Code Quality	Time Taken	Status
7	Java: Vending Machine Coding	Java (Intermediate)	75/75	-	33 min 49 sec	✓

1. Specification

Incorrect

Multiple Choice | Spring Boot | Hard

Question description

Which of the following options can be used for dynamic jpa queries with spring-data-jpa repositories?

Candidate's Solution

Options: (Expected answer indicated with a tick)

- ○ PreparedStatement
- ○ Specification ✓
- ○ Criteria
- ● Query

No comments.

2. Cracker

Correct

Multiple Choice | Spring Boot | Hard

Question description

Cracker.java

```
@Component
public class Cracker {
    public void startCracking(){
        System.out.println("Cracking...");
    }
    public void stopCracking(){
        System.out.println("Cracking Stopped!");
    }
}
```

CrackerService.java

```
@Service
public class CrackerService {

    @Autowired
    Cracker cracker;
    public void workWithNewCracker() {
        cracker.stopCracking();
        cracker = new Cracker();
    }
    public void startWorking() {
        cracker.startCracking();
    }
}
```

CrackerWorker.java

```
@Service
public class CrackerWorker {

    @Autowired
    CrackerService crackerService;

    public void startWorking() {
        crackerService.startWorking();
        crackerService.workWithNewCracker();
        crackerService = new CrackerService();
        crackerService.startWorking();
    }
}
```

Consider these classes are managed in Spring Boot. What happens when 'crackerWorker.startWorking()' is called in the Spring context?

Interviewer guidelines

CrackerService in CrackerWorker will work perfectly until it's replaced with new CrackerService. When that happens it just replace existing injected bean with empty pojo. Which doesn't have it's cracker object so when it's startWorking method is called it will try to execute null cracker attributes startWorking method and NullPointerException will arise.

Candidate's Solution

Options: (Expected answer indicated with a tick)

- ○ "Cracking..." "Cracking Stopped!" will be written to console
- ○ "Cracking..." "Cracking Stopped!" "Cracking..." will be written to console
- ● **"Cracking..." "Cracking Stopped!" NullPointerException will be thrown** ✓
- ○ None of above

No comments.

3. Output

Incorrect

Multiple Choice | Hard | Spring Boot | TDD

Question description

What will happen to the output file?

```
@SpringBootTest
public class LogTest
{
   File log;
   FileWriter logWriter;

   @Autowired
   SomeClassToTest someClassToTest;

   @Before
   public void init()
   {
     log = new File("log.txt");
     logWriter = new FileWriter(log);
   }

   @After
   public void teardown()
   {
      log.delete();
   }

   @Test
   public void test1()
   {
      logWriter.write("Entered Test 1");
      assertTrue(someClassToTest.someMethod());
   }

   @Test
   public void test2()
   {
      logWriter.write("Entered Test 2");
      assertTrue(someClassToTest.otherMethod());
   }
}
```

Candidate's Solution

Options: (Expected answer indicated with a tick)

- ○ The output file is not created.
- ● **The output file is created but not deleted.**
- ○ The output file is created and eventually deleted. ✓
- ○ The create and delete blocks will not be executed.

No comments.

4. Response

Correct

Multiple Choice | Hard | Spring Boot | TDD

Question description

Which response will cause the following test case to fail?

```
@Test
  @DisplayName("Filter Price Check 3")
  public void FilterPriceCheck3() throws Exception {
    mockMvc.perform(get("/filter/price/150/500"))
        .andDo(print())
```

```
            .andExpect(jsonPath("$[*].product", containsInAnyOrder("Product 1", "Product 2" , "Produc
t 5")))
    }
```

Candidate's Solution

Options: (Expected answer indicated with a tick)

- [{"product" : "Product 5"} , {"product" : "Product 1"}, {"product" : "Product 2"}]
- [{"product" : "Product 5"} , {"product" : "Product 2"}, {"product" : "Product 1"}]
- [{"product" : "Product 1"} , {"product" : "Product 2"}, {"product" : "Product 5"}]
- **[{"product" : "product 5"} , {"product" : "product 2"}, {"product" : "product 1"}]**

No comments.

5. Spring Scopes

Correct

Multiple Choice | Medium | Spring

Question description

Which of the following statements is/are true about @Scope("request") and @Scope("prototype") annotations?

Candidate's Solution

Options: (Expected answer indicated with a tick)

- [x] Both are used to define the scope of spring beans.
- [x] @Scope("request") creates a bean instance for a single HTTP request.
- [x] @RequestScope can be used instead of @Scope("request").
- [x] @Scope("prototype") will return a different instance every time it is requested from the container.

No comments.

---

6. Minimize Path Value

Correct

Coding Graphs Medium

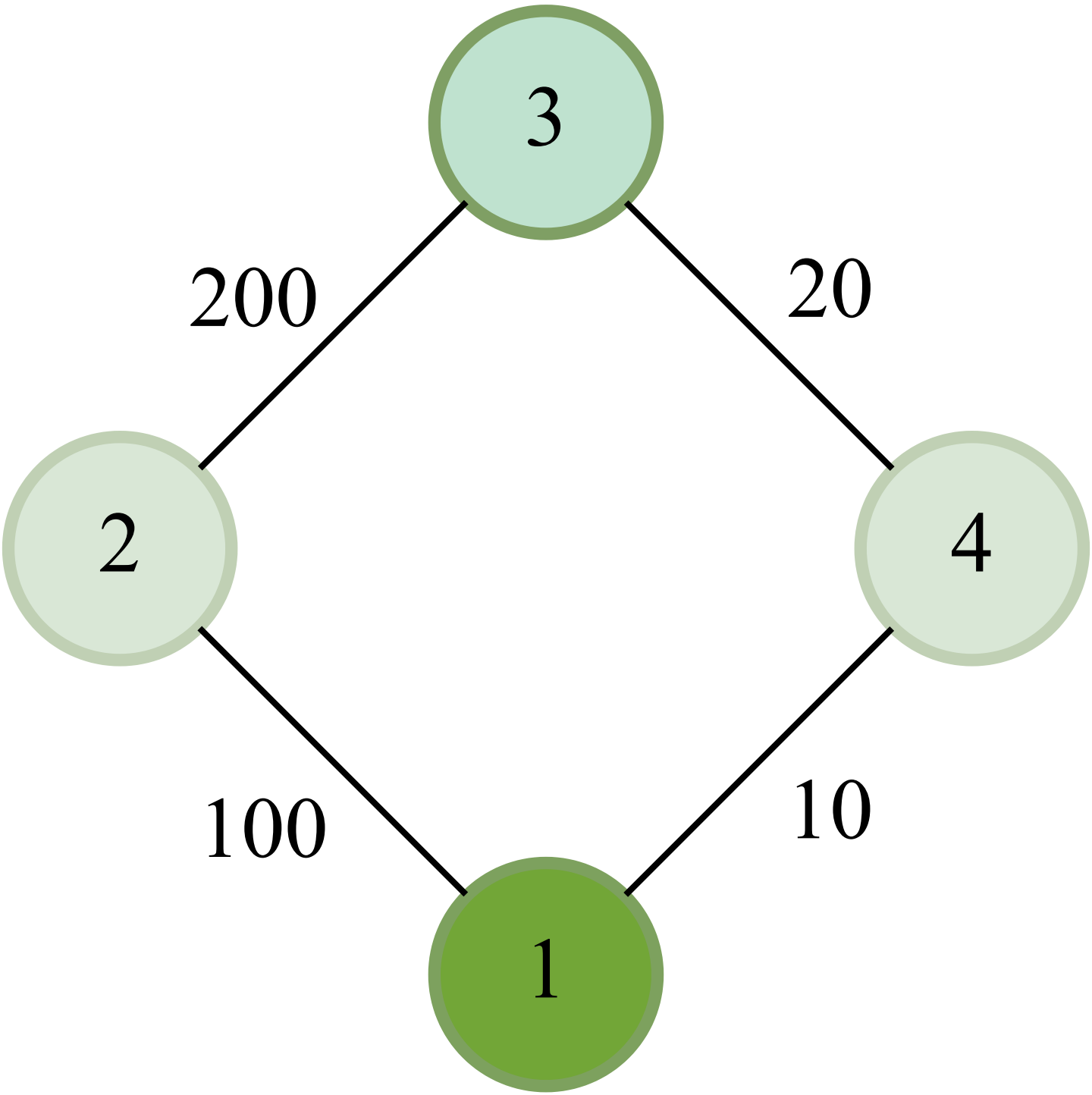
Question description

You have a weighted undirected graph with *graph_nodes* nodes and *m* edges. The stress level of a path between two nodes is defined as the weight of the heaviest edge in that path.

Given a source node *source* and a destination node *destination*, find the minimum possible stress level of a path. If no such path exists, return -1.

Example

Consider the following example below, where *source=1* and *destination=3.*



There are two paths, from node 1 to node 3.

1. The max weighted edge/stress level in path 1 -> 2 -> 3 is 200.
2. The max weighted edge/stress level in the path 1 -> 4 -> 3 is 20.

Return 20, the lower stress level from the second path.

Function Description

Complete the function *getMinimumStress* in the editor with the following parameter(s):

int graph_nodes: number of nodes
int graph_from[m]: one end node for an edge
int graph_to[m]: one end node for an edge
int graph_weight[m]: the weights of the edges
int source: the source node
int destination: the destination node

Returns

int: the value of the least stressful path, or -1 if no path exists

Note: Even though edge endpoints are called *graph_from[i]* and *graph_to[i]*, this is an undirected graph.

Constraints

- $1 \le graph_nodes \le 10^5$
- $1 \le |graph_from| = |graph_to| = |graph_weight| \le 10^5$ *(Where |A| denotes the size of array A)*
- $1 \le graph_from[i], graph_to[i], source, destination \le graph_nodes$
- $0 \le graph_weight[i] \le 10^9$

▼ INPUT FORMAT FOR CUSTOM TESTING

The first line contains two space-separated integers *graph_nodes* and *m,* the number of nodes, and the number of edges of the graph.
Each of the following *m* lines contains three space-separated integers *u, v,* and *w,* meaning there is an edge connecting the nodes *u* and *v* of weight *w.*
The next line contains a single integer, denoting the index of the "*source*" node.
The next line contains a single integer, denoting the index of the "*destination*" node.

▼ SAMPLE CASE 0

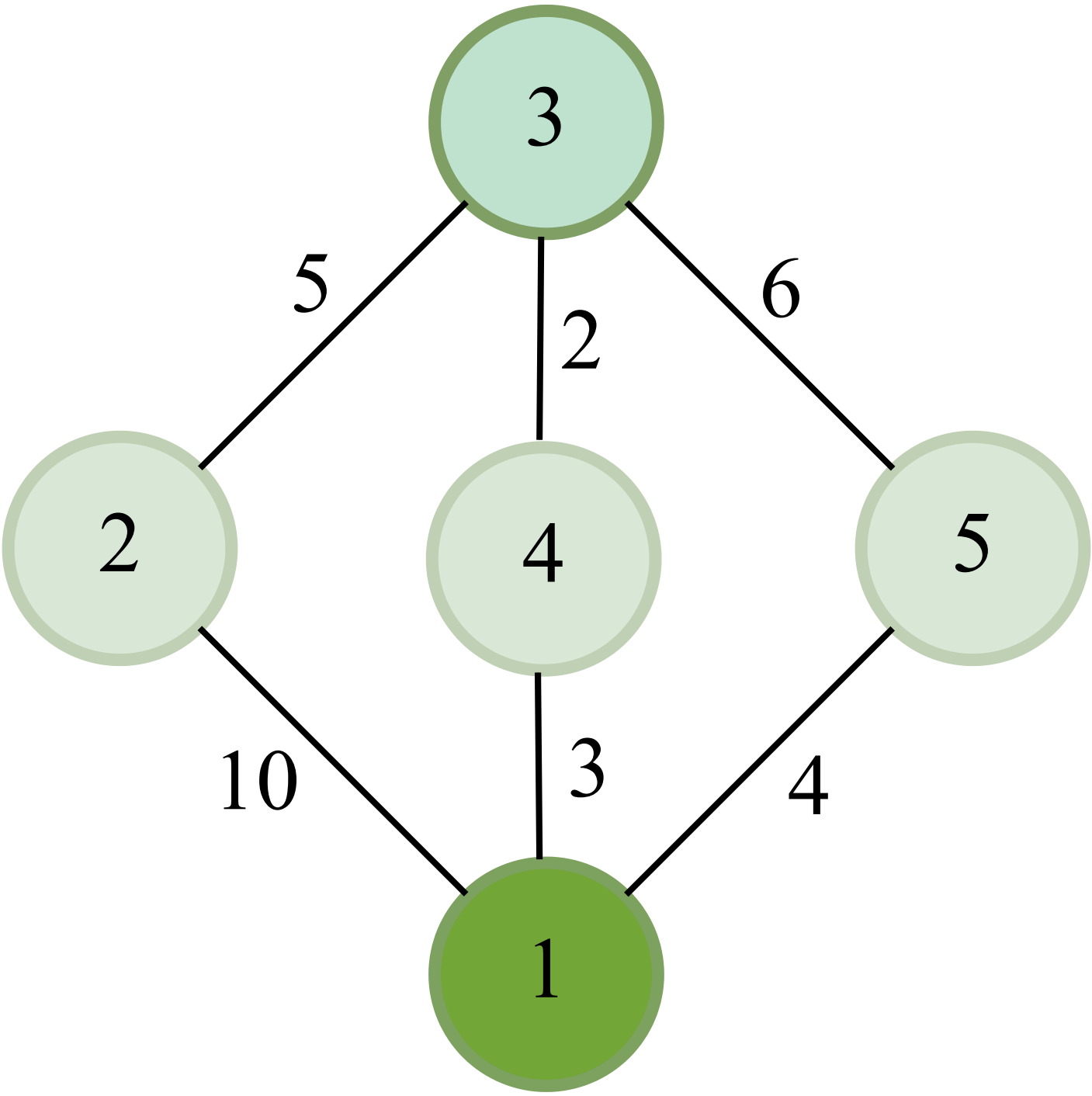
Sample Input For Custom Testing

```
STDIN       FUNCTION
-----       --------
5 6          graph_nodes = 5, m = 6
1 2 10       u = 1, v = 2, w = 10
2 3 5
1 4 3
4 3 2
1 5 4
5 3 6
1           source = 1
3           destination = 3
```

Sample Output

```
3
```

Explanation



There are three paths, from node 1 to node 3 :

1. 1 -> 2 -> 3. Edges have weights 10 and 5, the maximum weighted edge, and the stress level is 10.
2. 1 -> 4 -> 3. Edges have weights 3 and 2, and the stress level = 3.
3. 1 -> 5 -> 3. Edges have weights 4 and 6, and the stress level = 6.

▼ SAMPLE CASE 1

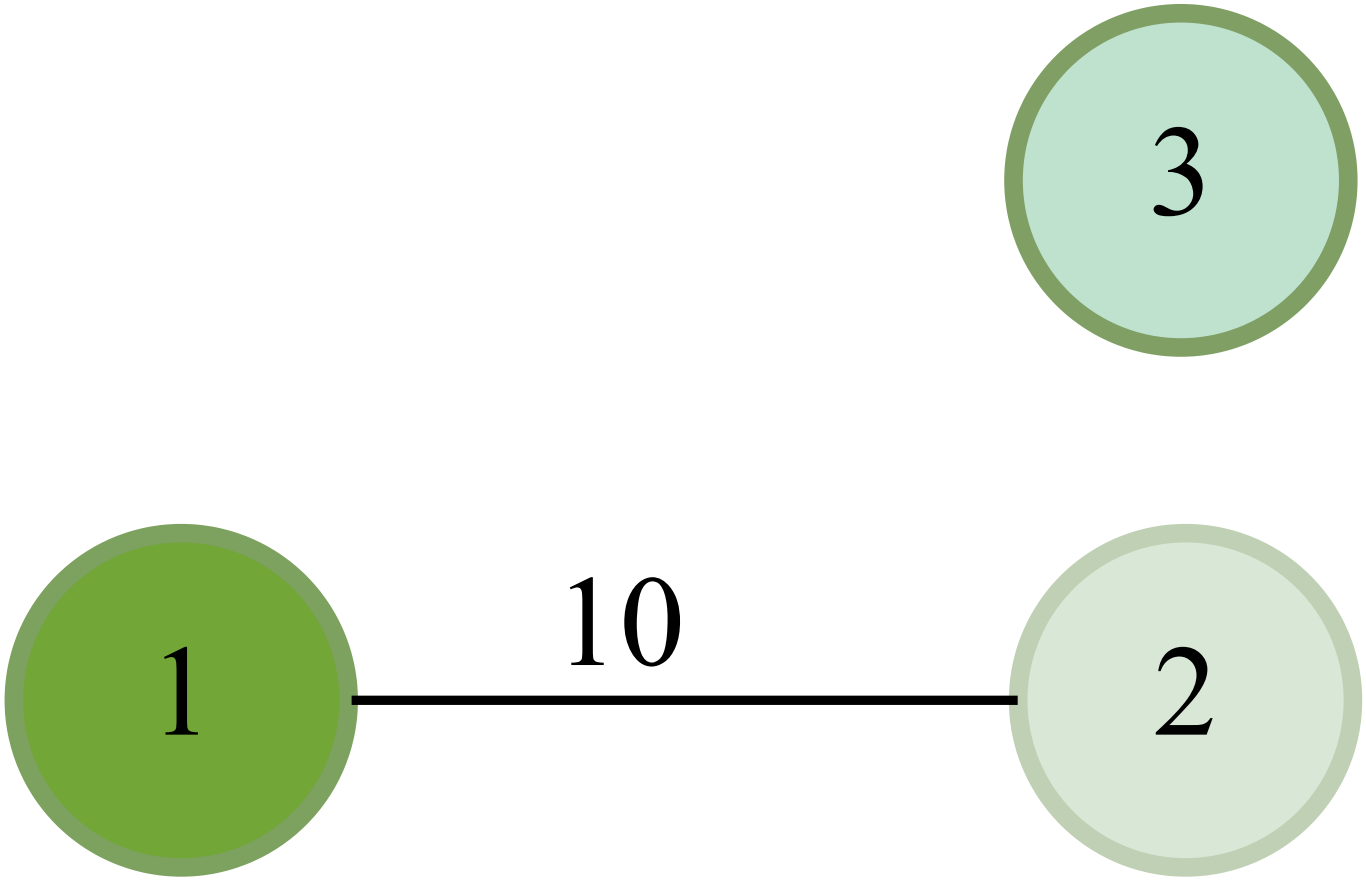
Sample Input For Custom Testing

```
STDIN       FUNCTION
-----       --------
3 1          graph_nodes = 3, m = 1
1 2 10       a = 1, b = 2, w = 10
1           source = 1
3           destination = 3
```

Sample Output

```
-1
```

Explanation



There is no path, from node 1 to node 3.

Interviewer guidelines

▼ SOLUTION

Python Solution

```
sys.setrecursionlimit(10**5)


def getMinimumStress(graph_nodes, graph_from, graph_to, graph_weight, source, destination):

    adj = [[] for i in range(graph_nodes + 1)]
    vis = [False] * (graph_nodes + 1)

    for x, y, w in zip(graph_from, graph_to, graph_weight):
        adj[x].append((y, w))
        adj[y].append((x, w))

    def dfs(node, mx):
        vis[node] = True
        if node == destination:
            return True
        ans = False
        for i, w in adj[node]:
            if w <= mx and not vis[i]:
                ans |= dfs(i, mx)
        return ans

    if not dfs(source, max(graph_weight)):
        return -1

    lo, hi = 0, max(graph_weight)
    while hi - lo > 1:
        vis = [False] * (graph_nodes + 1)
        mid = (hi + lo) // 2
        if dfs(source, mid):
            hi = mid
        else:
            lo = mid
    return hi
```

Candidate's Solution

Language used: **Python 3**

```python
#!/bin/python3

import math
import os
import random
import re
import sys
```

```python


#
# Complete the 'getMinimumStress' function below.
#
# The function is expected to return an INTEGER.
# The function accepts following parameters:
#  1. WEIGHTED_INTEGER_GRAPH graph
#  2. INTEGER source
#  3. INTEGER destination
#

#
# For the weighted graph, <name>:
#
# 1. The number of nodes is <name>_nodes.
# 2. The number of edges is <name>_edges.
# 3. An edge exists between <name>_from[i] and <name>_to[i]. The weight of the edge is <name>_weight[i].
#
#

def getMinimumStress(graph_nodes, graph_from, graph_to, graph_weight, source, destination):
    if source==destination:
        return 0
    edges=[]
    for i in range(len(graph_from)):
        edges.append((graph_weight[i] , graph_from[i] , graph_to[i]))
    edges.sort()
    parent = list(range(graph_nodes+1))
    rank = [0]*(graph_nodes+1)

    def find(i):
        root=i
        while parent[root]!=root:
            root=parent[root]
        while parent[i]!=root:
            next_node=parent[i]
            parent[i]=root
            i=next_node
        return root
    def union(i,j):
        ri=find(i)
        rj=find(j)

```

```
        if ri!=rj:
            if rank[ri] <rank[rj]:
                parent[ri]=rj
            elif rank[ri] > rank[rj]:
                parent[rj]=ri
            else:
                parent[ri]=rj
                rank[rj]+=1
            return True
        return False
    for i,j,k in edges:
        union(j,k)
        if find(source) == find(destination):
            return i

    return -1
if __name__ == '__main__':
    fptr = open(os.environ['OUTPUT_PATH'], 'w')

    graph_nodes, graph_edges = map(int, input().rstrip().split())

    graph_from = [0] * graph_edges
    graph_to = [0] * graph_edges
    graph_weight = [0] * graph_edges

    for i in range(graph_edges):
        graph_from[i], graph_to[i], graph_weight[i] = map(int, input().rstrip().split())

    source = int(input().strip())

    destination = int(input().strip())

    result = getMinimumStress(graph_nodes, graph_from, graph_to, graph_weight, source, destination)

    fptr.write(str(result) + '\n')

    fptr.close()

```

TESTCASE	DIFFICULTY	TYPE	STATUS	SCORE	TIME TAKEN	MEMORY USED

TestCase 0	Easy	Sample	Success	1	0.0267 sec	10.6 KB
TestCase 1	Easy	Sample	Success	1	0.0257 sec	10.5 KB
TestCase 2	Easy	Sample	Success	1	0.0264 sec	10.6 KB
TestCase 3	Easy	Hidden	Success	6	0.4255 sec	29.3 KB
TestCase 4	Easy	Hidden	Success	6	0.3406 sec	26.7 KB
TestCase 5	Easy	Hidden	Success	6	0.4006 sec	30.6 KB
TestCase 6	Easy	Hidden	Success	6	0.3689 sec	33.9 KB
TestCase 7	Easy	Hidden	Success	6	0.3685 sec	34.8 KB
TestCase 8	Easy	Hidden	Success	6	0.3779 sec	28.3 KB
TestCase 9	Easy	Hidden	Success	6	0.3037 sec	30.9 KB
TestCase 10	Easy	Hidden	Success	6	0.351 sec	31.6 KB
TestCase 11	Easy	Hidden	Success	6	0.4634 sec	30.9 KB
TestCase 12	Easy	Hidden	Success	6	0.1591 sec	18.9 KB

TestCase 13	Easy	Hidden	Success	6	0.231 sec	22 KB
TestCase 14	Easy	Hidden	Success	6	0.3804 sec	32.8 KB

No comments.

7. Java: Vending Machine

Correct

Coding Medium Java Exceptions OOP

Question description

Design a vending machine that serves 3 drinks: "Coke", "Fanta", and "Sprite".

Create a *VendingMachine* class that implements the following methods:

- *void registerDrink(int buttonNumber, Drink drink)*: Registers a type of drink with a particular *buttonNumber* of the vending machine. Here, *Drink* is an interface provided in the locked stub.
- *ServedDrinkSummary(int buttonNumber, int money)*: Returns the Drink that is served when *buttonNumber* is pressed, and an amount of money is entered by a customer.

The class contains two attributes:

- *change*: the units of currency to return to the consumer
- *drink*: the drink requested

There are three potential error conditions:

- If the machine has run out of the drink, an *OutOfStockException* should be thrown with the message "{drinkName} is out of stock".
- If the money the user enters is insufficient to buy the drink, an *InsufficientMoneyException* should be thrown.
- If the drink is out of stock and the user has not entered enough money, throw an *OutOfStockException* with the previously mentioned message.

The *OutOfStockException, InsufficientMoneyException*, and *ServedDrinkSummary* classes are provided in the locked stub.

The locked stub validates the *VendingMachine* class by processing some requests.

Constraints

- $1 \leq totalNumberOfRequests \leq 10^4$
- $0 \leq buttonNumber \leq 2$
- $0 \leq money \leq 100$
- $0 \leq quantity$ available of each $drink \leq 10^4$
- $0 \leq price$ per unit of each $drink \leq 10^4$
- No two drinks are assigned the same *buttonNumber.*

▼ INPUT FORMAT FOR CUSTOM TESTING

The first line contains three integers that represent the *quantity* of *Coke*, *Fanta,* and *Sprite* in the vending machine respectively.
The second line contains three integers that represent the *price* per unit of *Coke*, *Fanta,* and *Sprite* respectively.
The third line contains three integers that represent the *buttonNumbers* assigned to *Coke*, *Fanta,* and *Sprite* respectively.
The fourth line contains an integer, *totalNumberOfRequests .*
The next *totalNumberOfRequests* lines contain the *buttonNumber* pressed and *money* inserted separated by a space.

▼ SAMPLE CASE 0

Sample Input For Custom Testing

```
STDIN    Function
-----    --------
2 1 0    quantity Coke = 2, Fanta = 1, Sprite = 0
5 10 15  price Coke = 5, Fanta = 10, Sprite = 15
0 1 2    buttonNumbers Coke = 0, Fanta = 1, Sprite = 2
4        totalNumberOfRequests = 4
0 20     buttonNumber = 0, money = 20 for the first request
2 20     buttonNumber = 2, money = 20 for the second request
2 5      ...
1 5
```

Sample Output

```
Vending machine set up
Coke 15
Sprite is out of stock
Sprite is out of stock
Insufficient money
```

Explanation

- *0 20:* A request to process a can of *Coke* and *20* units of money is provided to the vending machine. *Coke* is in stock and costs *5* units of money so *15* units in change is returned to the consumer. The locked stub prints "*Coke 15".*
- *2 20:* A request to process a can of *Sprite* and *20* units of money is provided to the vending machine. *Sprite* is out of stock so an *OutOfStockException* is thrown, and the stub prints "*Sprite is out of stock".*
- *2 5:* A request to process a can of *Sprite* and *5* units of money is provided to the vending machine. *Sprite* is out of stock and the amount of *money* is insufficient so an *OutOfStockException* is thrown and the stub prints, "*Sprite is out of stock".*
- *1 5:* A request to process a can of *Fanta* and *5* units of money is provided. Since the price of a unit of *Fanta is 10* and it is in stock, an *InsufficientMoneyException* is thrown, and the stub prints "*Insufficient money".*

Interviewer guidelines

▼ SOLUTION

Solution for Java 7, 8 and 15

```
class VendingMachine{
    HashMap<Integer, Drink> drinkRelatedToButton = new HashMap<>();

    void registerDrink(int buttonIdx, Drink drink){
        drinkRelatedToButton.put(buttonIdx, drink);
    }

    ServeDrinkSummary dispatch(int buttonPressed, int money)throws OutOfStockException, Insuffi
cientMoneyException{
        ServeDrinkSummary serveDrinkSummary = new ServeDrinkSummary();
        serveDrinkSummary.servedDrink = drinkRelatedToButton.get(buttonPressed);
        serveDrinkSummary.change = money - serveDrinkSummary.servedDrink.getPrice();
```

```
        if(serveDrinkSummary.servedDrink.getQuantityLeft() == 0){
            throw new OutOfStockException(serveDrinkSummary.servedDrink.getName() + " is out of st
ock");
        }

        if(serveDrinkSummary.change < 0){
            throw new InsufficientMoneyException();
        }
        return serveDrinkSummary;
    }
}
```

Candidate's Solution

Language used: **Java 21**

```java
import java.io.*;
import java.util.*;
import java.text.*;
import java.math.*;
import java.util.regex.*;

interface Drink{
    int getPrice();
    String getName();
    int getQuantityLeft();
    void serveDrink();
}

class ServeDrinkSummary{
    Drink servedDrink;
    int change;
}

class OutOfStockException extends Exception{
    public OutOfStockException(String message){
        super(message);
    }
}

class InsufficientMoneyException extends Exception{
    public InsufficientMoneyException(){
        super();
    }
}

class VendingMachine{
```

```
    private Map<Integer,Drink> map = new HashMap<>();

    void registerDrink(int buttonIdx, Drink drink){
        this.map.put(buttonIdx, drink);
    }


    ServeDrinkSummary dispatch(int buttonPressed, int money)
        throws OutOfStockException, InsufficientMoneyException{
            Drink drink = map.get(buttonPressed);
            if (drink.getQuantityLeft()<=0)
                throw new OutOfStockException(drink.getName()+" is out of stock");

            if(drink.getPrice() >money)
                throw new InsufficientMoneyException();

            ServeDrinkSummary summary= new ServeDrinkSummary();
            summary.change=money- drink.getPrice();
            summary.servedDrink=drink;
            return summary;
    }
}

class Solution{

    public static void main(String[] args) throws IOException{
        String arr[];
        int quantity[] = new int[3], pricePerUnit[] = new int[3], buttonAssigned[] = new int[3];

        BufferedReader br = new BufferedReader(new InputStreamReader(System.in));
        PrintWriter out = new PrintWriter(System.out);

        arr = br.readLine().split(" ");
        for(int i = 0; i < 3; i++){
            quantity[i] = Integer.parseInt(arr[i]);
        }

        arr = br.readLine().split(" ");
        for(int i = 0; i < 3; i++){
            pricePerUnit[i] = Integer.parseInt(arr[i]);
        }

        arr = br.readLine().split(" ");
```

```
        for(int i = 0; i < 3; i++){
            buttonAssigned[i] = Integer.parseInt(arr[i]);
        }

        VendingMachine vendingMachine = new VendingMachine();
        Drink coke = new Coke(pricePerUnit[0], "Coke", quantity[0]);
        Drink fanta = new Fanta(pricePerUnit[1], "Fanta", quantity[1]);
        Drink sprite = new Sprite(pricePerUnit[2], "Sprite", quantity[2]);
        vendingMachine.registerDrink(buttonAssigned[0], coke);
        vendingMachine.registerDrink(buttonAssigned[1], fanta);
        vendingMachine.registerDrink(buttonAssigned[2], sprite);
        out.println("Vending machine set up");

        int totalNumberOfRequests = Integer.parseInt(br.readLine().trim());
        while(totalNumberOfRequests-- > 0){
            arr = br.readLine().split(" ");
            int buttonPressed = Integer.parseInt(arr[0]),
                money = Integer.parseInt(arr[1]);
            try{
                ServeDrinkSummary serveDrinkSummary = vendingMachine.dispatch(buttonPressed, money);
                serveDrinkSummary.servedDrink.serveDrink();
                out.println(serveDrinkSummary.servedDrink.getName() + " " + serveDrinkSummary.change);
            }catch(OutOfStockException e){
                out.println(e.getMessage());
            }catch(InsufficientMoneyException e){
                out.println("Insufficient money");
            }
        }

        out.flush();
        out.close();
    }
}

class Coke implements Drink{
    private int price;
    private String name;
    private int quantityLeft;

    public Coke(int price, String name, int quantityLeft){
        this.price = price;
        this.name = name;
        this.quantityLeft = quantityLeft;
    }
```

```java

    public int getPrice(){
        return this.price;
    }

    public String getName(){
        return this.name;
    }

    public int getQuantityLeft(){
        return this.quantityLeft;
    }

    public void serveDrink(){
        if(this.quantityLeft > 0)
            this.quantityLeft -= 1;
    }
}

class Fanta implements Drink{
    private int price;
    private String name;
    private int quantityLeft;

    public Fanta(int price, String name, int quantityLeft){
        this.price = price;
        this.name = name;
        this.quantityLeft = quantityLeft;
    }

    public int getPrice(){
        return this.price;
    }

    public String getName(){
        return this.name;
    }

    public int getQuantityLeft(){
        return this.quantityLeft;
    }

    public void serveDrink(){
        if(this.quantityLeft > 0)
            this.quantityLeft -= 1;
    }
```

```
}

class Sprite implements Drink{
    private int price;
    private String name;
    private int quantityLeft;

    public Sprite(int price, String name, int quantityLeft){
        this.price = price;
        this.name = name;
        this.quantityLeft = quantityLeft;
    }

    public int getPrice(){
        return this.price;
    }

    public String getName(){
        return this.name;
    }

    public int getQuantityLeft(){
        return this.quantityLeft;
    }

    public void serveDrink(){
        if(this.quantityLeft > 0)
            this.quantityLeft -= 1;
    }
}
```

TESTCASE	DIFFICULTY	TYPE	STATUS	SCORE	TIME TAKEN	MEMORY USED
TestCase 0	Easy	Sample	Success	1	0.0527 sec	37.5 KB
TestCase 1	Easy	Hidden	Success	6	0.0422 sec	37.1 KB

TestCase 2	Easy	Hidden	Success	6	0.073 sec	39.1 KB
TestCase 3	Easy	Hidden	Success	6	0.0713 sec	38.4 KB
TestCase 4	Medium	Hidden	Success	10	0.0768 sec	39.1 KB
TestCase 5	Medium	Hidden	Success	10	0.1659 sec	50.4 KB
TestCase 6	Medium	Hidden	Success	10	0.1145 sec	50.7 KB
TestCase 7	Hard	Hidden	Success	13	0.14 sec	50.2 KB
TestCase 8	Hard	Hidden	Success	13	0.1307 sec	52.1 KB

No comments.